

LAB 1

DevOps Foundations & Continuous Integration

Task 6 - Implement a Basic Jenkins CI/CD Pipeline

Student: **Kunal**

Objective

Implement a basic Jenkins CI/CD pipeline that pulls source code from a GitHub repository and triggers an automated build. The evidence demonstrates Jenkins job creation, Pipeline configuration, GitHub source checkout, automated build execution, and a successful pipeline result.

Jenkins Job: Kunal-Lab1-Basic-CICD

Source Repository: lab1-devops-version-control

Branch: main

1. CI/CD Workflow

The implemented workflow follows this sequence:

1. GitHub Repository	2. Jenkins Checkout	3. Build Stage	4. Result
Source code stored in GitHub	Jenkins pulls main branch	Automated build commands execute	Pipeline finishes: SUCCESS

Pipeline implementation

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url:
'https://github.com/kunalprajapati14042005/lab1-devops-version-control.git'
            }
        }

        stage('Build') {
            steps {
                sh 'echo "Automated build completed successfully."'
                sh 'ls -la'
            }
        }
    }
}
```

This Pipeline definition contains two required functional parts: **Checkout** retrieves the source from GitHub, while **Build** performs an automated build operation inside the Jenkins workspace.

2. Jenkins New Item screen

A Jenkins Pipeline job named **Kunal-Lab1-Basic-CICD** is created, demonstrating the use of Jenkins Pipeline for automated build and CI/CD execution.

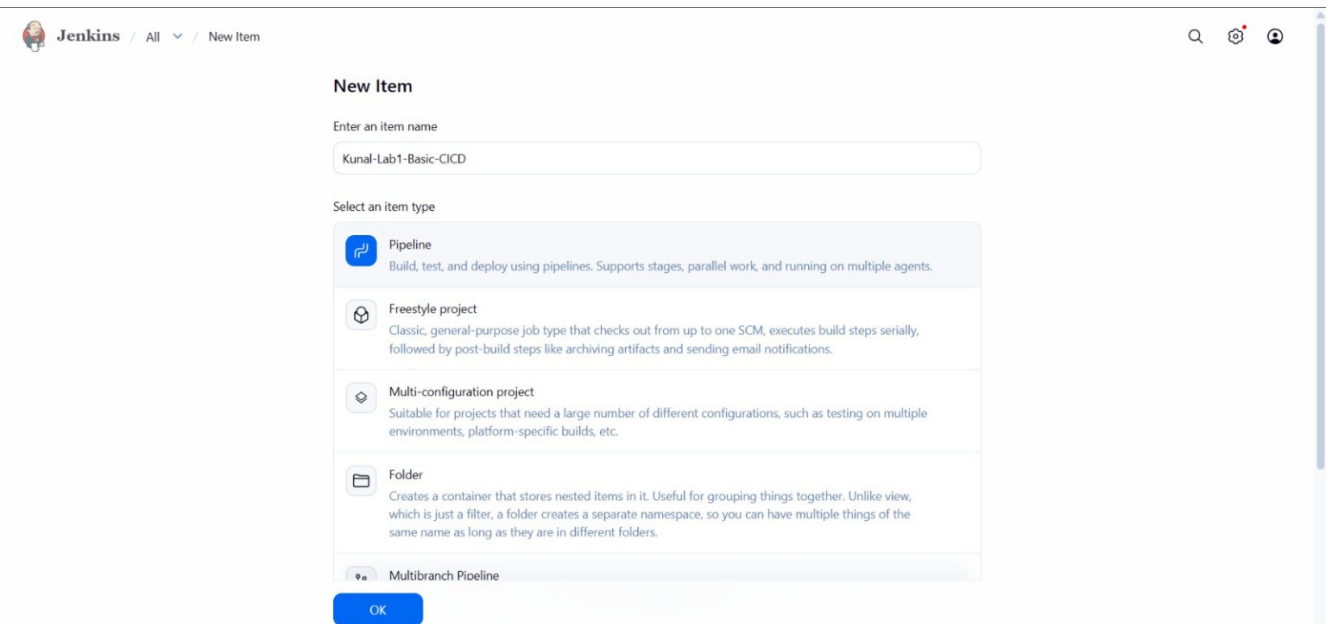


Figure 1. Jenkins New Item screen.

3. Pipeline configuration with GitHub checkout and build stages

The Pipeline script checks out the **main** branch from the GitHub repository and then executes an automated Build stage. The checkout URL and build commands are visible in the Jenkins configuration.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
5 stage('checkout') {
6     steps {
7         git branch: 'main',
8           url: 'https://github.com/kunalprajapati14042005/lab1-devops-version-control.git'
9     }
10 }
11
12 stage('Build') {
13     steps {
14         sh 'echo "Automated build completed successfully."'
15         sh 'ls -la'
16     }
17 }
18 }
19 }
```

try sample Pipeline...

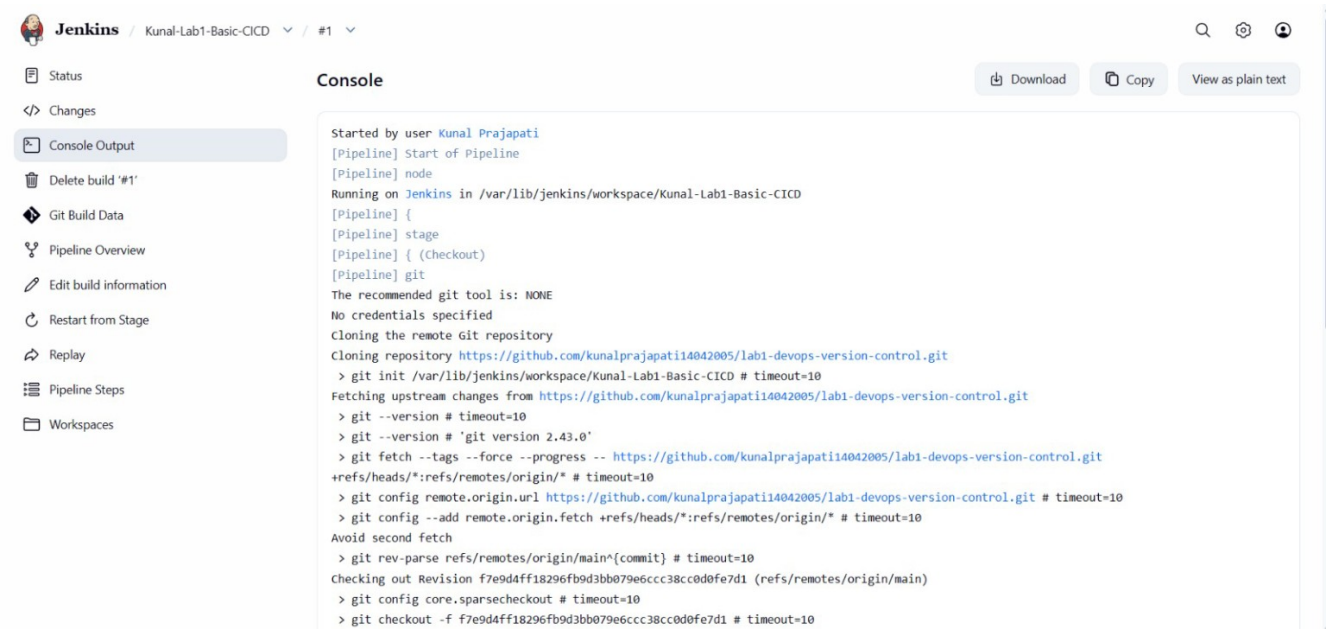
Save

Apply

Figure 2. Pipeline configuration with GitHub checkout and build stages.

4. Jenkins Build #1 console output

Jenkins successfully clones/fetches the GitHub repository, checks out the required revision, executes the Build stage, runs the automated build command, lists the workspace contents, and finishes with **SUCCESS**.



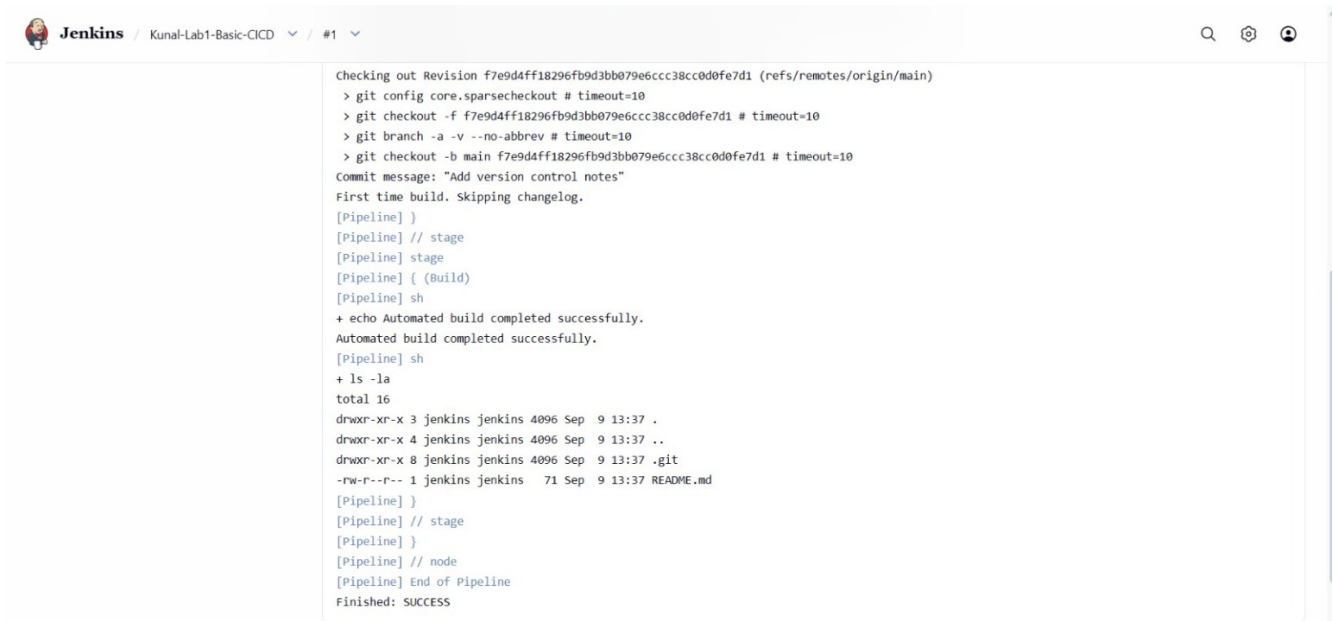
The screenshot shows the Jenkins web interface for a build named 'Kunal-Lab1-Basic-CICD'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#1', Git Build Data, Pipeline Overview, Edit build information, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The main area is titled 'Console' and displays the build's log output. At the top right of the console area are buttons for 'Download', 'Copy', and 'View as plain text'. The log output shows the build starting with 'Started by user Kunal Prajapati', followed by '[Pipeline] Start of Pipeline' and '[Pipeline] node'. It then indicates the build is running on 'Jenkins' in the workspace '/var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD'. The pipeline script is shown, including a 'stage' block with a '((Checkout))' step. The checkout step uses 'git' to clone the repository 'https://github.com/kunalprajapati14042005/lab1-devops-version-control.git'. The log shows the git command sequence: 'git init', 'git fetch --tags --force --progress -- https://github.com/kunalprajapati14042005/lab1-devops-version-control.git +refs/heads/*:refs/remotes/origin/*', 'git config remote.origin.url https://github.com/kunalprajapati14042005/lab1-devops-version-control.git', 'git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/*', 'git rev-parse refs/remotes/origin/main^{commit}', and 'git checkout -f f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1'. The log ends with 'Checking out Revision f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 (refs/remotes/origin/main)'.

```
Started by user Kunal Prajapati
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD
[Pipeline] {
[Pipeline] stage
[Pipeline] { ((Checkout))
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git init /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD # timeout=10
Fetching upstream changes from https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/kunalprajapati14042005/lab1-devops-version-control.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/kunalprajapati14042005/lab1-devops-version-control.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
```

Figure 3. Jenkins Build #1 console output.

5. Successful Jenkins build and GitHub checkout evidence

The console output confirms the pipeline ran on Jenkins, retrieved the source from GitHub, executed the automated build, and completed successfully. This provides direct execution evidence for the basic CI/CD pipeline.



```
Jenkins / Kunal-Lab1-Basic-CICD / #1

Checking out Revision f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
Commit message: "Add version control notes"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ echo Automated build completed successfully.
Automated build completed successfully.
[Pipeline] sh
+ ls -la
total 16
drwxr-xr-x 3 jenkins jenkins 4096 Sep  9 13:37 .
drwxr-xr-x 4 jenkins jenkins 4096 Sep  9 13:37 ..
drwxr-xr-x 8 jenkins jenkins 4096 Sep  9 13:37 .git
-rw-r--r-- 1 jenkins jenkins  71 Sep  9 13:37 README.md
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 4. Successful Jenkins build and GitHub checkout evidence.

6. Requirement Mapping and Result

Task 6 requirement	Evidence / result
Pull source code from GitHub	Checkout stage uses the GitHub repository URL and main branch.
Implement a Jenkins CI/CD pipeline	Pipeline job Kunal-Lab1-Basic-CICD is configured as a Jenkins Pipeline.
Trigger an automated build	Build stage runs automatically when the Jenkins job is executed.
Demonstrate execution	Jenkins Console Output shows checkout and Build stage execution.
Successful result	Console output ends with Finished: SUCCESS .

Conclusion

Task 6 was completed successfully. Jenkins is configured to retrieve source code from the GitHub main branch and execute an automated Build stage. The captured console evidence confirms successful source checkout, build execution, and a final **SUCCESS** result.